

ЛАБОРАТОРНА РОБОТА

Microsoft Power BI: Python-скрипти та моделі машинного навчання для візуалізації

Знахур С.В.

1. Мета роботи

Метою даної лабораторної роботи є практичне освоєння середовища Microsoft Power BI для виконання аналізу даних із застосуванням мови програмування Python. Студент навчиться інтегрувати Python-скрипти у Power BI для створення складних візуалізацій та побудови моделей машинного навчання — включаючи регресію, дерева рішень та кластеризацію — безпосередньо у звітах Power BI.

Після виконання роботи студент повинен:

1. Розуміти архітектуру інтеграції Python та Power BI Desktop.
2. Вміти налаштовувати Python-середовище та встановлювати необхідні бібліотеки.
3. Будувати власні Python-візуалізації на основі даних Power BI.
4. Навчати та застосовувати моделі машинного навчання (лінійна регресія, дерево рішень, K-means) безпосередньо у Power BI.
5. Завантажувати збережені ML-моделі та використовувати їх для прогнозування у Power BI.
6. Аналізувати результати візуалізацій та метрики якості моделей.

2. Теоретичні відомості

2.1. Інтеграція Python та Power BI

Microsoft Power BI підтримує виконання Python-скриптів у двох режимах: як джерело даних (Python script у меню Get Data) та як візуальний елемент (Python visual на панелі Visualizations). У обох випадках Power BI передає дані у вигляді pandas DataFrame з назвою dataset, а результатом виконання скрипту є або новий DataFrame (для трансформацій), або графік matplotlib (для візуалізацій).

2.2. Ключові бібліотеки Python для Power BI

Бібліотека	Призначення	Команда встановлення
pandas	Маніпуляції з даними, DataFrame	pip install pandas
matplotlib	Базова побудова графіків	pip install matplotlib
seaborn	Статистичні візуалізації	pip install seaborn
scikit-learn	ML-моделі: регресія, дерева, кластеризація	pip install scikit-learn
numpy	Числові обчислення	pip install numpy
joblib	Збереження та завантаження ML-моделей	pip install joblib
plotly	Інтерактивні графіки (обмежена підтримка)	pip install plotly

2.3. Типи ML-моделей, що розглядаються

Тип моделі	Scikit-learn клас	Задача	Вхідні дані
Лінійна регресія	LinearRegression	Прогноз числового значення	Числові ознаки → числова мітка
Дерево рішень (класиф.)	DecisionTreeClassifier	Класифікація на категорії	Ознаки → категорія
Дерево рішень (регресія)	DecisionTreeRegressor	Прогноз числового значення	Ознаки → число
K-means кластеризація	KMeans	Unsupervised групування	Числові ознаки (без міток)
Random Forest	RandomForestClassifier	Ансамблева класифікація	Ознаки → категорія

3. Підготовка середовища

3.1. Встановлення Python та бібліотек

Якщо Python ще не встановлено, завантажте його з <https://www.python.org/> або через Microsoft Store. Після встановлення відкрийте командний рядок (cmd) та виконайте наступні команди для встановлення всіх необхідних бібліотек.

Power BI підтримує Python версії 3.x. Anaconda та Miniconda також підтримуються.

```
# Встановлення всіх необхідних бібліотек одночасно
pip install pandas matplotlib seaborn scikit-learn numpy joblib

# Перевірка встановлення
python -c "import pandas, matplotlib, sklearn; print('OK')"
```

3.2. Налаштування Power BI Desktop для Python

Щоб Power BI міг виконувати Python-скрипти, необхідно вказати шлях до інтерпретатора:

1. Відкрийте Power BI Desktop.
2. Перейдіть до меню Файл → Параметри та налаштування → Параметри.
3. У лівому стовпці виберіть розділ Сценарії Python (Python scripting).
4. У полі «Домашній каталог Python» натисніть Огляд та виберіть папку, яка містить python.exe (наприклад, C:\Users\User\AppData\Local\Programs\Python\Python311).
5. Натисніть ОК для збереження налаштувань.

Щоб знайти шлях до python.exe, виконайте у cmd команду: where python. Скопіюйте шлях без "python.exe" у кінці.

3.3. Завантаження та підключення даних

У цій лабораторній роботі використовуються три файли: car.csv, customer.csv, sale.csv. Для завантаження виконайте:

6. На стрічці Home натисніть Get Data → Text/CSV.

- Виберіть файл car.csv і натисніть Load. Повторіть для customer.csv та sale.csv.
- Перейдіть у вкладку Model view та створіть зв'язки між таблицями: cars[vin] → sales[vin] та customers[dealer_id] → sales[dealer_id].

4. Приклади Python-візуалізацій у Power BI

4.1. Побудова базового графіка розподілу

Для додавання Python-візуалізації виконайте наступні кроки:

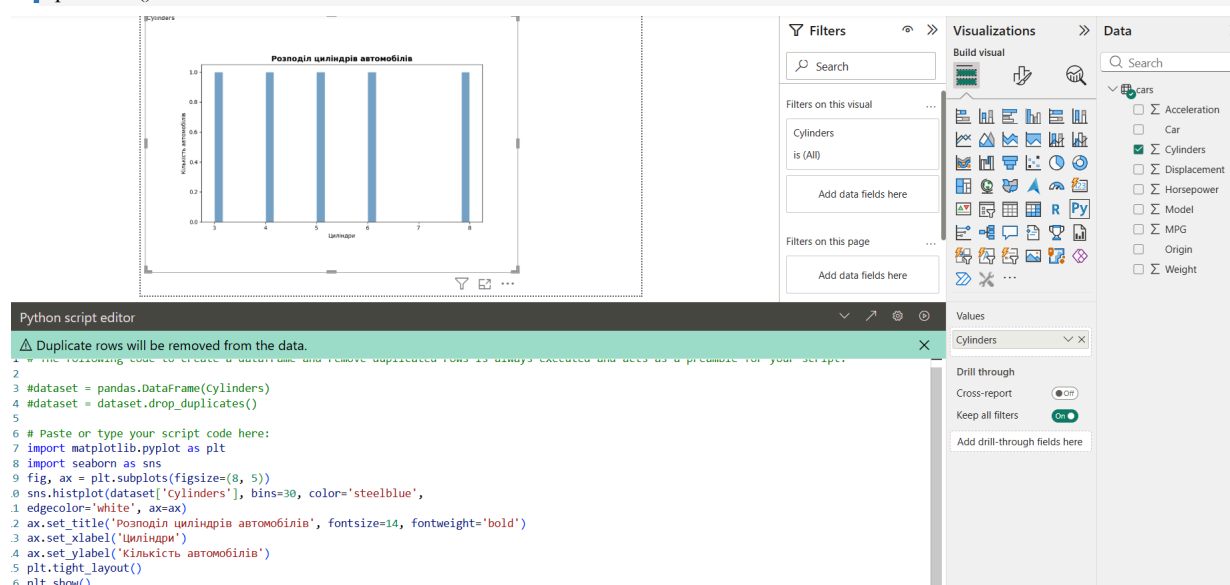
- На панелі Visualizations натисніть іконку Py (Python visual).
- У вікні Enable script visuals натисніть Enable.
- З панелі Fields перетягніть потрібні поля у розділ Values візуалізації.
- У редакторі Python script editor напишіть код.
- Натисніть кнопку Run (▶) для виконання скрипту.

Приклад: гістограма розподілу пробігу автомобілів

Перетягніть поле mileage у Values. Введіть код у редактор:

```
# dataset — це DataFrame, автоматично наданий Power BI
# Містить лише ті стовпці, які додані у Values
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(8, 5))
sns.histplot(dataset['mileage'], bins=30, color='steelblue',
             edgecolor='white', ax=ax)
ax.set_title('Розподіл пробігу автомобілів', fontsize=14, fontweight='bold')
ax.set_xlabel('Пробіг (милі)')
ax.set_ylabel('Кількість автомобілів')
plt.tight_layout()
plt.show()
```



4.2. Діаграма розсіювання: пробіг vs ціна продажу

Перетягніть поля mileage та purchase_price (або sale_price) у Values. Скрипт:

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

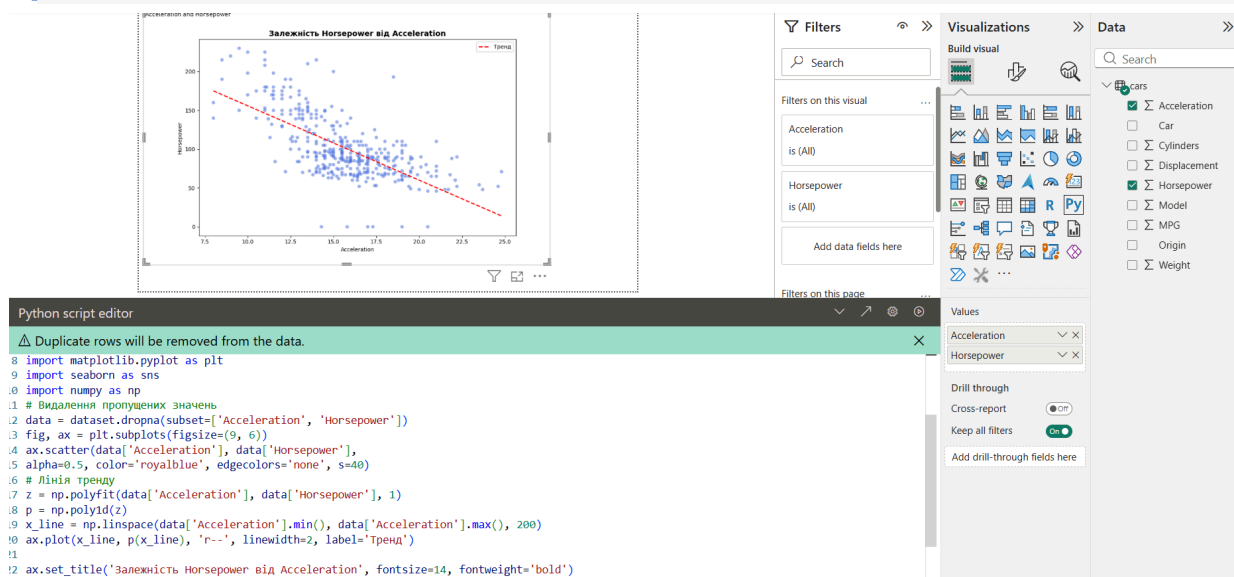
# Видалення пропущених значень
data = dataset.dropna(subset=['mileage', 'purchase_price'])

fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(data['mileage'], data['purchase_price'],
           alpha=0.5, color='royalblue', edgecolors='none', s=40)

# Лінія тренду
z = np.polyfit(data['mileage'], data['purchase_price'], 1)
p = np.poly1d(z)
x_line = np.linspace(data['mileage'].min(), data['mileage'].max(), 200)
ax.plot(x_line, p(x_line), 'r--', linewidth=2, label='Тренд')

ax.set_title('Залежність ціни від пробігу', fontsize=14, fontweight='bold')
ax.set_xlabel('Пробіг (милі)')
ax.set_ylabel('Ціна ($)')
ax.legend()
plt.tight_layout()
plt.show()

```



5. Навчання ML-моделей у Power BI

5.1. Лінійна регресія: прогноз ціни за пробігом

Лінійна регресія — це метод, що знаходить лінійну залежність між незалежними змінними (ознаками) та залежною змінною (цільовою). У цьому прикладі ми прогнозуємо purchase_price на основі mileage.

Перетягніть поля mileage та purchase_price у Values. Код скрипту:

```

import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Підготовка даних

```

```

data = dataset.dropna(subset=['mileage', 'purchase_price'])
X = data[['mileage']] # ознака (2D масив)
y = data['purchase_price'] # цільова змінна

# Розбивка на тренувальну та тестову вибірки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Навчання моделі
model = LinearRegression()
model.fit(X_train, y_train)

# Оцінка якості
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Візуалізація
fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(X_test, y_test, alpha=0.5, color='steelblue',
           label='Реальні значення', s=30)
x_range = np.linspace(X['mileage'].min(), X['mileage'].max(), 300).reshape(-1,1)
ax.plot(x_range, model.predict(x_range), 'r-', linewidth=2,
       label=f'Регресія (R²={r2:.3f})')
ax.set_title(f'Лінійна регресія: пробіг → ціна\nRMSE={rmse:.0f}$, R²={r2:.3f}',
            fontsize=13, fontweight='bold')
ax.set_xlabel('Пробіг (милі)')
ax.set_ylabel('Ціна ($)')
ax.legend()
plt.tight_layout()
plt.show()

```

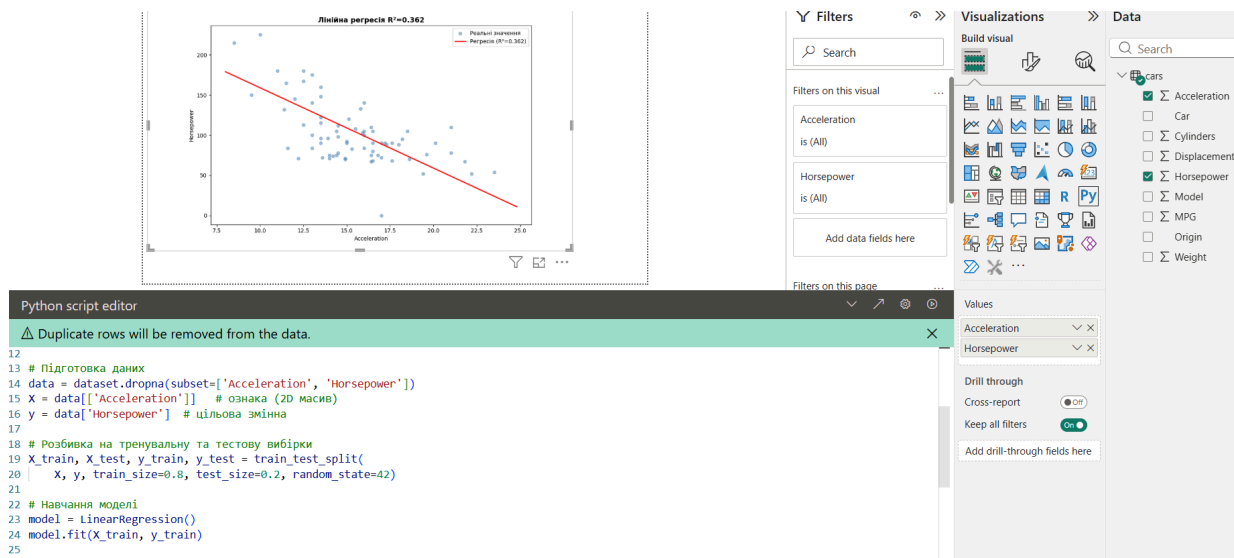
Пояснення:

`train_test_split` — розбиває дані на тренувальну (80%) та тестову (20%) вибірки.
`random_state=42` забезпечує відтворюваність результатів.

`LinearRegression().fit()` — навчає модель, знаходячи оптимальні коефіцієнти методом найменших квадратів.

`r2_score` — коефіцієнт детермінації R^2 . Значення 1.0 — ідеальна відповідність; 0.0 — модель не краща за середнє.

`mean_squared_error` з `squared=False` — RMSE (корінь із середньоквадратичної похибки) у тих же одиницях, що й цільова змінна.



5.2. Дерево рішень: класифікація кольору автомобіля

Дерево рішень (Decision Tree) — алгоритм, що будує ієрархічну структуру правил для класифікації або регресії. У цьому прикладі ми класифікуємо автомобіль за кольором на основі числових характеристик.

Перетягніть поля mileage, purchase_price та color у Values. Код:

```

import matplotlib.pyplot as plt
import numpy as np
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder

# Підготовка даних
data = dataset.dropna(subset=['mileage', 'purchase_price', 'color'])

# Кодування цільової категорії (текст → числа)
le = LabelEncoder()
y = le.fit_transform(data['color'])
X = data[['mileage', 'purchase_price']]

# Розбивка та навчання
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X_train, y_train)

# Оцінка
acc = accuracy_score(y_test, clf.predict(X_test))

# Візуалізація дерева
fig, ax = plt.subplots(figsize=(14, 7))
plot_tree(clf, feature_names=['mileage', 'purchase_price'],
          class_names=le.classes_, filled=True, rounded=True,
          fontsize=8, ax=ax)
ax.set_title(f'Дерево рішень: класифікація кольору\nТочність: {acc:.1%}',
            fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

```

Пояснення:

- LabelEncoder — перетворює текстові категорії (кольори) на числові коди (0, 1, 2...). Це необхідно, оскільки scikit-learn працює лише з числами.
- max_depth=4 — обмежує глибину дерева для уникнення перенавчання. Без обмеження дерево може ідеально підходити до тренувальних даних, але погано узагальнювати.
- stratify=y — гарантує, що в тренувальній та тестовій вибірках представлені всі класи пропорційно.
- plot_tree з filled=True — малює дерево з кольоровим кодуванням: темніший колір означає більшу «чистоту» (переважання одного класу).

5.3. K-means кластеризація

K-means — алгоритм unsupervised навчання, що розбиває дані на K кластерів без наявності міток. Алгоритм ітеративно присвоює точки до найближчого центроїда та оновлює центроїди.

Перетягніть поля mileage та purchase_price у Values. Код:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Підготовка та нормалізація даних
data = dataset.dropna(subset=['mileage', 'purchase_price'])
X = data[['mileage', 'purchase_price']].values

# Стандартизація (важливо для K-means!)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Вибір оптимальної кількості кластерів (метод ліктя)
inertias = []
K_range = range(2, 8)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

# Навчання з оптимальним K=3
km_final = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = km_final.fit_predict(X_scaled)

# Візуалізація: 2 subplot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Метод ліктя
ax1.plot(list(K_range), inertias, 'bo-', markersize=8)
ax1.axvline(x=3, color='red', linestyle='--', label='Обраний K=3')
ax1.set_title('Метод ліктя для вибору K')
ax1.set_xlabel('Кількість кластерів K')
ax1.set_ylabel('Інерція (Within-cluster SS)')
ax1.legend()

# Кластери
colors = ['#e41a1c', '#377eb8', '#4daf4a']
for i in range(3):
    mask = labels == i
    ax2.scatter(X[mask, 0], X[mask, 1],
                c=colors[i], label=f'Кластер {i+1}')
```

```

alpha=0.6, s=30)

# Центроїди (повертаємо у вихідний масштаб)
centers_orig = scaler.inverse_transform(km_final.cluster_centers_)
ax2.scatter(centers_orig[:, 0], centers_orig[:, 1],
            c='black', marker='X', s=200, zorder=5, label='Центроїди')
ax2.set_title('K-means кластеризація (K=3)')
ax2.set_xlabel('Пробіг (милі)')
ax2.set_ylabel('Ціна ($)')
ax2.legend()
plt.tight_layout()
plt.show()

```

Пояснення:

- StandardScaler — нормалізує ознаки до нульового середнього та одиничного відхилення. K-means чутливий до масштабу: без нормалізації ознака з більшим діапазоном домінуватиме.
- Метод ліктя (Elbow method) — графік інерції при різних K. Оптимальний K знаходиться у точці, де крива «ломається» — додавання ще одного кластера дає незначне покращення.
- n_init=10 — K-means запускається 10 разів із різними початковими центроїдами; обирається найкращий результат.
- inverse_transform — повертає координати центроїдів у вихідний масштаб для зрозумілої інтерпретації на графіку.

6. Збереження та завантаження ML-моделей

6.1. Навчання та збереження моделі поза Power BI

Для виробничих сценаріїв рекомендується навчати модель окремо (у Jupyter Notebook або Python-скрипті), зберігати її у файл, а потім завантажувати у Power BI для прогнозів. Це дає переваги: швидше виконання скрипту, версіонування моделей, розширені налаштування навчання.

Крок 1: Навчання та збереження моделі (виконати в Jupyter або Python-скрипті):

```

# train_and_save_model.py — виконати ПОЗА Power BI
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import joblib
import os

# 1. Завантаження даних
cars = pd.read_csv('car.csv')
sales = pd.read_csv('sale.csv')
df = pd.merge(cars, sales, on='vin')
df = df.dropna(subset=['mileage', 'purchase_price', 'color'])

# 2. Підготовка ознак
X = df[['mileage', 'purchase_price']]
le = LabelEncoder()
y = le.fit_transform(df['color'])

```

```

# 3. Навчання Random Forest
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# 4. Оцінка
print(classification_report(y_test, rf.predict(X_test),
    target_names=le.classes_))

# 5. Збереження моделі та енкодера
save_dir = r'C:\PowerBI_Models'
os.makedirs(save_dir, exist_ok=True)
joblib.dump(rf, os.path.join(save_dir, 'rf_color_model.pkl'))
joblib.dump(le, os.path.join(save_dir, 'label_encoder.pkl'))
print('Модель збережена у', save_dir)

```

6.2. Завантаження збереженої моделі у Power BI

Крок 2: Завантаження моделі у Python visual (виконати у Power BI):

Перетягніть поля mileage та purchase_price у Values. Введіть код:

```

# Скрипт завантаження та застосування збереженої моделі
import matplotlib.pyplot as plt
import numpy as np
import joblib
import os

# Шлях до збережених моделей
MODEL_DIR = r'C:\PowerBI_Models'

# Завантаження моделі та енкодера
rf = joblib.load(os.path.join(MODEL_DIR, 'rf_color_model.pkl'))
le = joblib.load(os.path.join(MODEL_DIR, 'label_encoder.pkl'))

# Підготовка даних із Power BI
data = dataset.dropna(subset=['mileage', 'purchase_price'])
X_new = data[['mileage', 'purchase_price']]

# Прогнозування
y_pred = rf.predict(X_new)
y_proba = rf.predict_proba(X_new).max(axis=1)
data = data.copy()
data['predicted_color'] = le.inverse_transform(y_pred)
data['confidence'] = y_proba

# Візуалізація: розподіл прогнозованих кольорів
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

# Кругова діаграма прогнозів
color_counts = data['predicted_color'].value_counts()
ax1.pie(color_counts.values, labels=color_counts.index,
    autopct='%1.1f%%', startangle=90)
ax1.set_title('Розподіл прогнозованих кольорів')

# Розподіл впевненості моделі
ax2.hist(data['confidence'], bins=20, color='forestgreen', edgecolor='white')
ax2.axvline(data['confidence'].mean(), color='red', linestyle='--',
    label=f'Середня впевненість: {data["confidence"].mean():.1%}')
ax2.set_title('Впевненість моделі у прогнозах')
ax2.set_xlabel('Ймовірність')
ax2.set_ylabel('Кількість')

```

```
ax2.legend()
plt.suptitle('Random Forest: завантажена модель', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

joblib є кращим інструментом для збереження scikit-learn моделей, ніж pickle, оскільки ефективніше серіалізує великі NumPy масиви всередині моделей.

6.3. Збереження та завантаження моделі кластеризації

Аналогічний підхід працює для будь-якого scikit-learn estimator. Приклад для K-means:

```
# --- НАВЧАННЯ (поза Power BI) ---
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import joblib

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X_scaled)

# Зберігаємо і модель, і scaler (важливо!)
joblib.dump(km, 'kmeans_model.pkl')
joblib.dump(scaler, 'scaler.pkl')

# --- ЗАВАНТАЖЕННЯ (у Power BI Python visual) ---
import joblib
km = joblib.load(r'C:\PowerBI_Models\kmeans_model.pkl')
scaler = joblib.load(r'C:\PowerBI_Models\scaler.pkl')

data = dataset.dropna(subset=['mileage', 'purchase_price'])
X = data[['mileage', 'purchase_price']].values
X_scaled = scaler.transform(X) # transform, не fit_transform!
labels = km.predict(X_scaled)
data = data.copy()
data['cluster'] = labels
```

При завантаженні використовуйте scaler.transform(), а не scaler.fit_transform(). fit_transform перенавчить scaler на нових даних, що призведе до некоректних прогнозів.

Завдання по варіантах

Кожен студент виконує індивідуальне завдання на базі датасету автомобілів (car.csv, sale.csv, customer.csv). Всі варіанти включають підготовку даних, Python-візуалізацію та побудову ML-моделі.

Підготовка середовища та даних

Встановіть Python та всі необхідні бібліотеки.

Налаштуйте Power BI Desktop для роботи з Python.

Завантажте файли car.csv, customer.csv, sale.csv та створіть зв'язки між таблицями.

Створіть Python visual із базовою гістограмою будь-якого числового поля.

Індивідуальні варіанти

Варіант	ML-завдання	Ознаки (Features)	Цільова змінна (Target)
1	Лінійна регресія + Кластеризація (K=4)	mileage, year (рік)	purchase_price; кластери за mileage + price
2	Дерево рішень (класифікація) + Регресія	mileage, purchase_price	color (клас кольору); mileage (прогноз)
3	Random Forest + K-means	purchase_price, mileage	color (класифікація); K=3 кластери авто
4	Лінійна регресія + Дерево рішень	year, purchase_price	mileage (регресія); color (дерево рішень)
5	K-means + Лінійна регресія	mileage, purchase_price, year	K=4 кластери; purchase_price за mileage+year

Опис індивідуальних завдань

Варіант 1. Лінійна регресія + K-means кластеризація

Завдання 1: Побудуйте лінійну регресію для прогнозу ціни автомобіля за пробігом та роком випуску (множинна регресія). Порівняйте точність моделі з однофакторною регресією (лише mileage). Виведіть коефіцієнти регресії та їх інтерпретацію.

Завдання 2: Виконайте K-means кластеризацію (K=4) за ознаками mileage та purchase_price. Побудуйте візуалізацію кластерів та поясніть характеристику кожного кластера (наприклад: «кластер 1 — дорогі нові авто з малим пробігом»).

Варіант 2. Дерево рішень + Лінійна регресія

Завдання 1: Навчіть DecisionTreeClassifier для класифікації кольору автомобіля (target: color) на основі mileage та purchase_price. Обмежте глибину дерева до max_depth=5. Візуалізуйте дерево та виведіть важливість ознак (feature_importances_).

Завдання 2: Побудуйте лінійну регресію для прогнозу пробігу mileage на основі purchase_price. Перевірте гіпотезу: чи можна по ціні автомобіля передбачити його пробіг?

Варіант 3. Random Forest + K-means

Завдання 1: Навчіть RandomForestClassifier (100 дерев) для класифікації кольору автомобіля за purchase_price та mileage. Порівняйте точність Random Forest з DecisionTreeClassifier (з розділу 5.2). Побудуйте confusion matrix для оцінки помилок по кожному класу.

Завдання 2: Виконайте K-means (K=3) та збережіть навчену модель у файл за допомогою joblib. Потім завантажте її та застосуйте до нового підмножини даних (dataset.sample(50)).

Варіант 4. Лінійна регресія + Дерево рішень

Завдання 1: Побудуйте лінійну регресію для прогнозу mileage (пробіг) на основі year (рік) та purchase_price. Проаналізуйте residuals (залишки) — побудуйте графік реальних vs прогнозованих значень та гістограму залишків.

Завдання 2: Навчіть DecisionTreeClassifier для передбачення кольору автомобіля. Попередньо збережіть модель у файл за допомогою joblib, а потім завантажте її у Power BI Python visual для прогнозування.

Варіант 5. K-means + Множинна лінійна регресія

Завдання 1: Виконайте K-means кластеризацію (K=4) за трьома ознаками: mileage, purchase_price та умовним year. Оскільки кластеризація трьохмірна, для візуалізації використайте 2D-проекцію або PCA. Опишіть характеристику кожного кластера.

Завдання 2: Побудуйте множинну лінійну регресію для прогнозу purchase_price на основі mileage та year. Додатково порівняйте моделі через cross-validation.

Контрольні питання

Що таке Python visual у Power BI? Яким чином Power BI передає дані у Python-скрипт?

Які бібліотеки Python потрібні для роботи з Power BI? Яка з них є обов'язковою для побудови графіків?

Поясніть різницю між навчанням моделі (fit) та її застосуванням (predict/transform) у scikit-learn.

Що таке train_test_split? Навіщо розбивати дані на тренувальну та тестову вибірки?

Поясніть метрики R^2 та RMSE для оцінки регресійної моделі. Що означають їх граничні значення?

Що таке LabelEncoder? Чому не можна використовувати текстові категорії безпосередньо в scikit-learn?

Поясніть параметр max_depth у DecisionTreeClassifier. Що відбувається при max_depth=1 та max_depth=None?

Що таке StandardScaler? Чому K-means потребує нормалізації даних?

Яка різниця між joblib.dump() та joblib.load()? Чому потрібно зберігати і модель, і scaler разом?

Що означає параметр n_estimators у RandomForestClassifier? Як він впливає на якість та швидкість?

Поясніть поняття перенавчання (overfitting) та недонавчання (underfitting). Як їх виявити у Power BI?